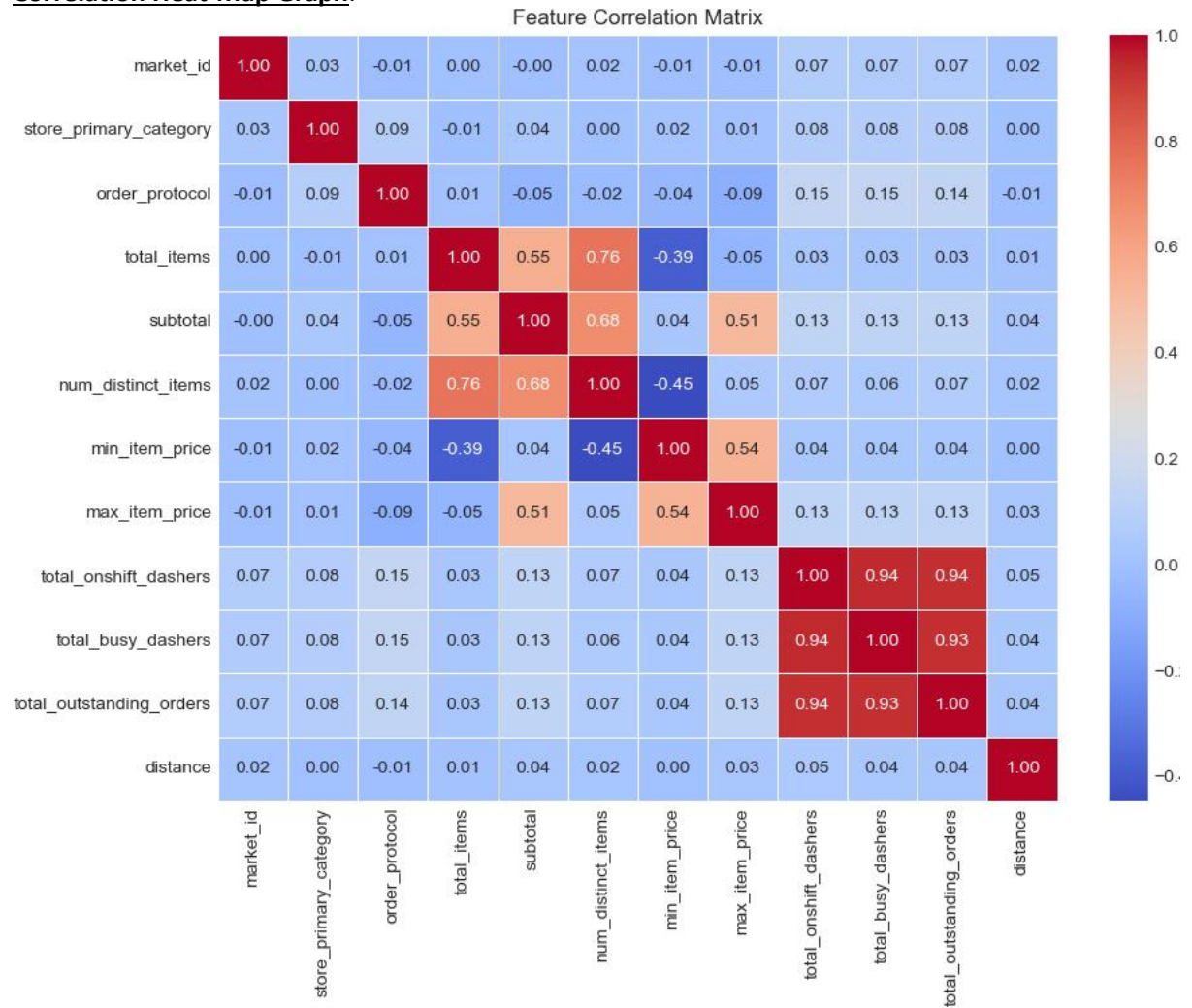
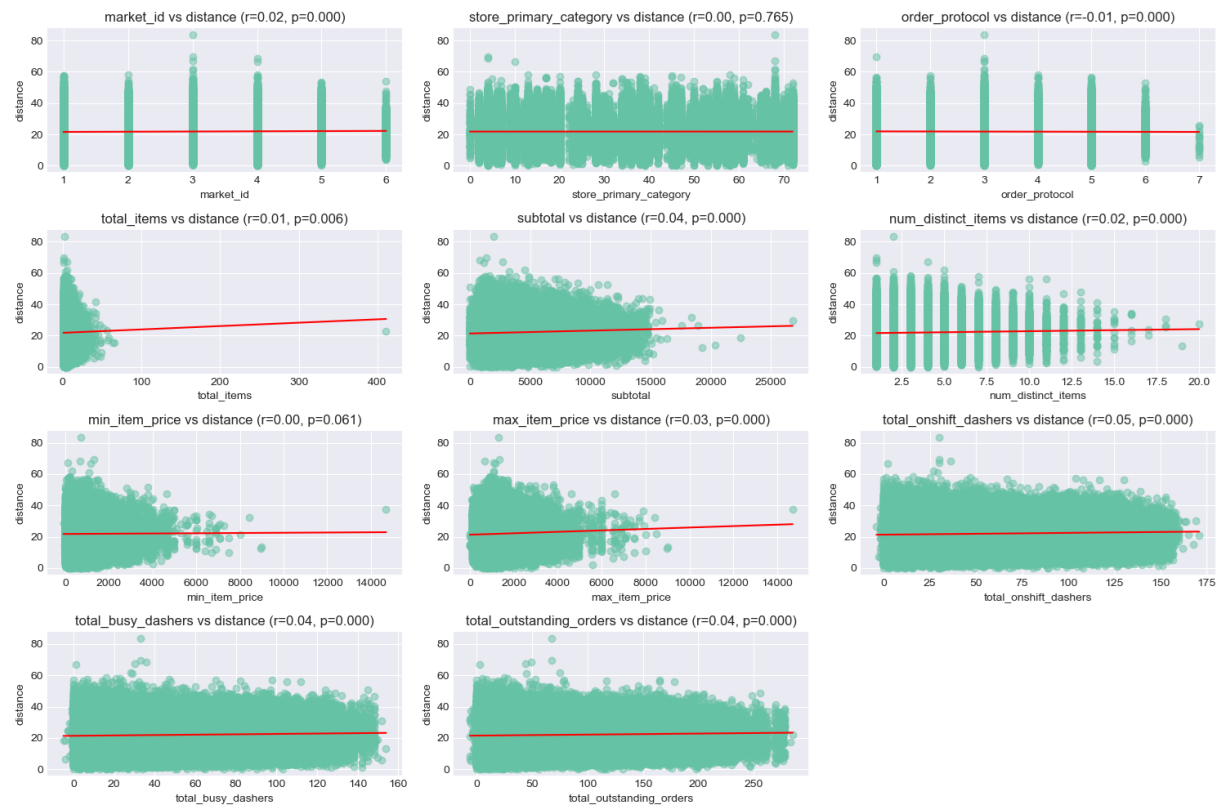


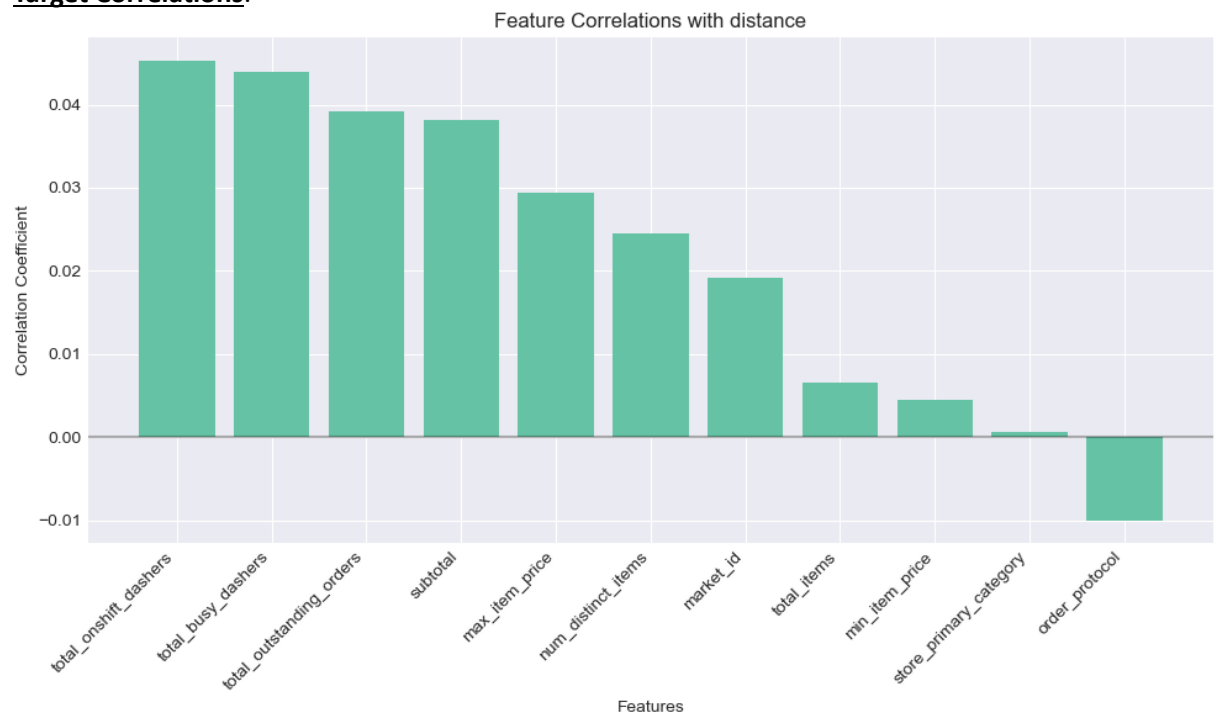
1) Correlation Heat-Map Graph:



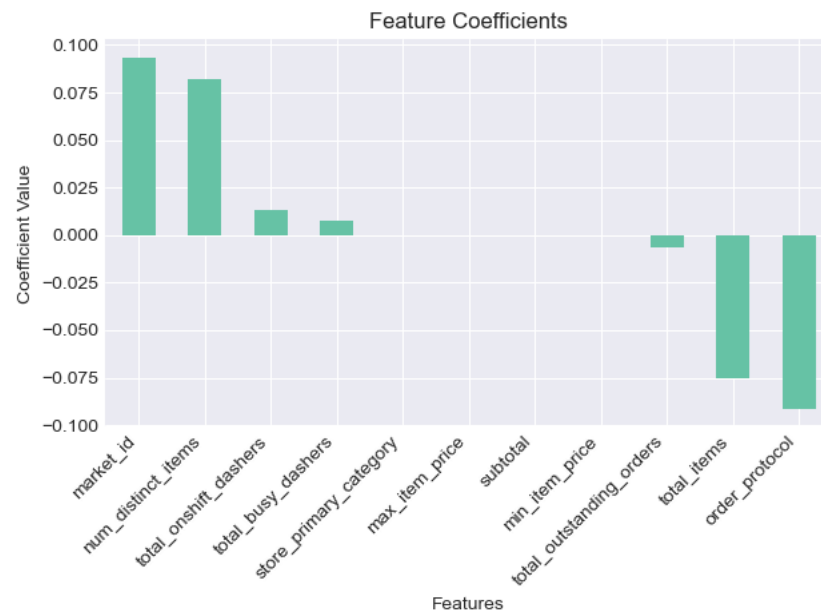
2) Feature Relationships:



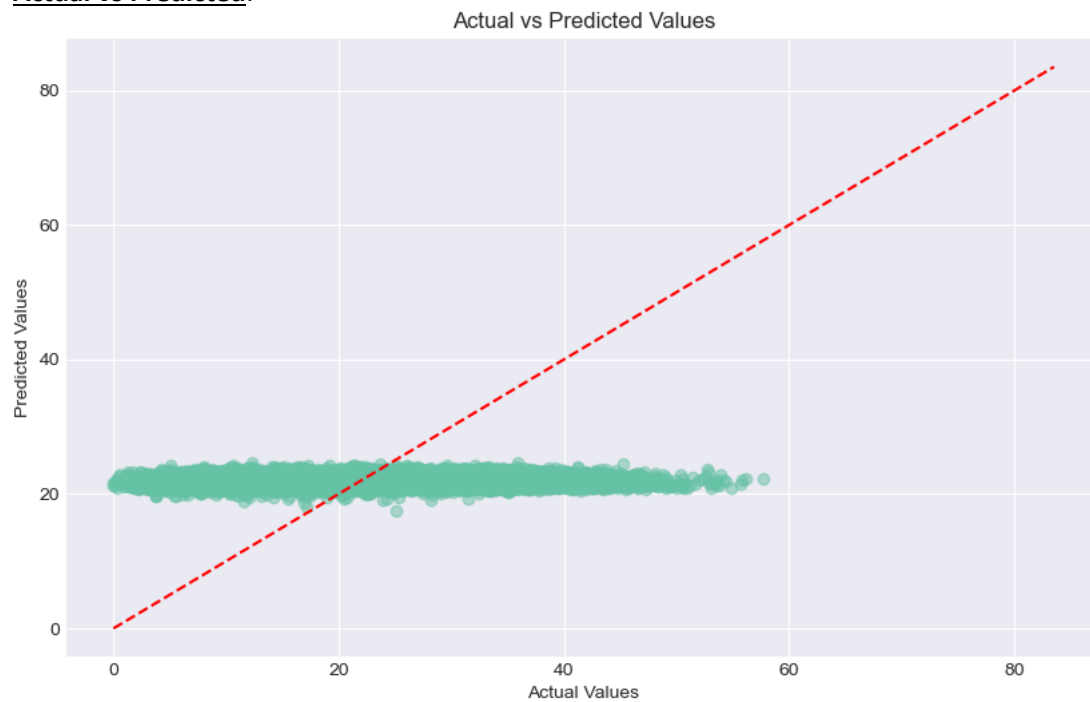
3) Target Correlations:



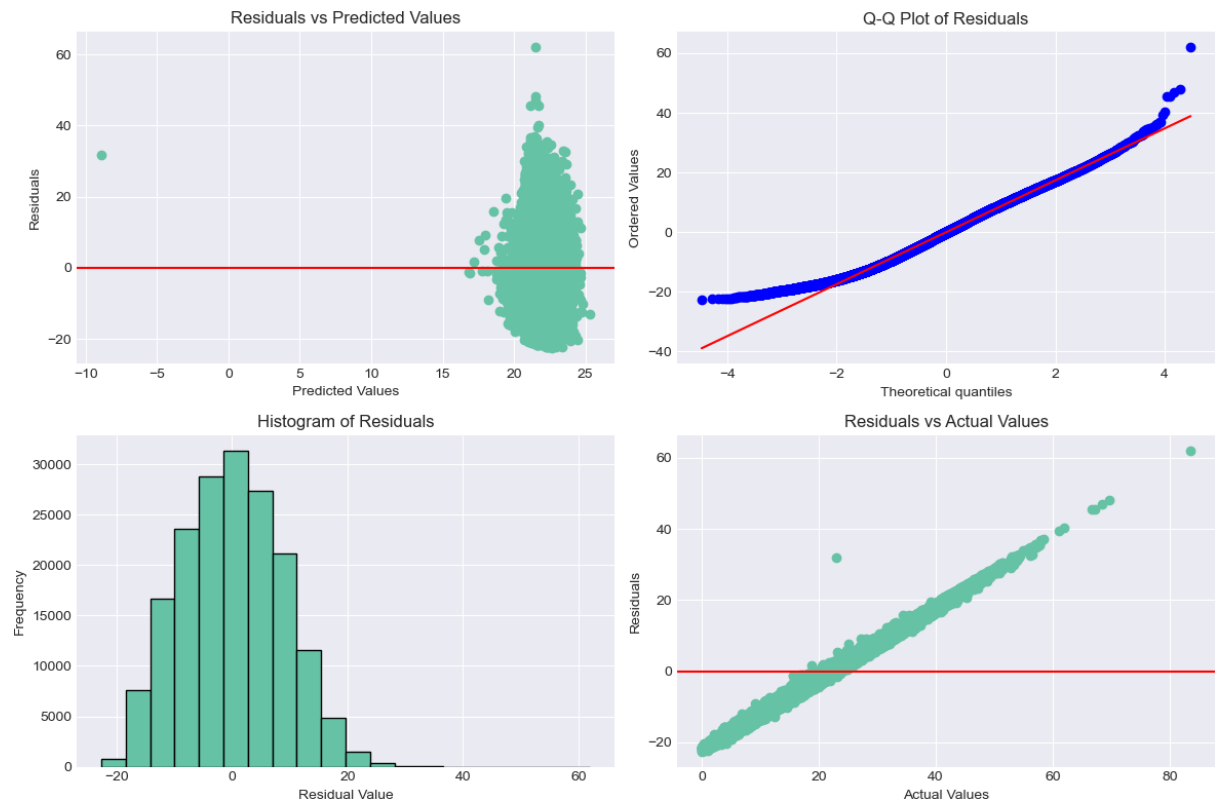
4) Feature Coefficients:



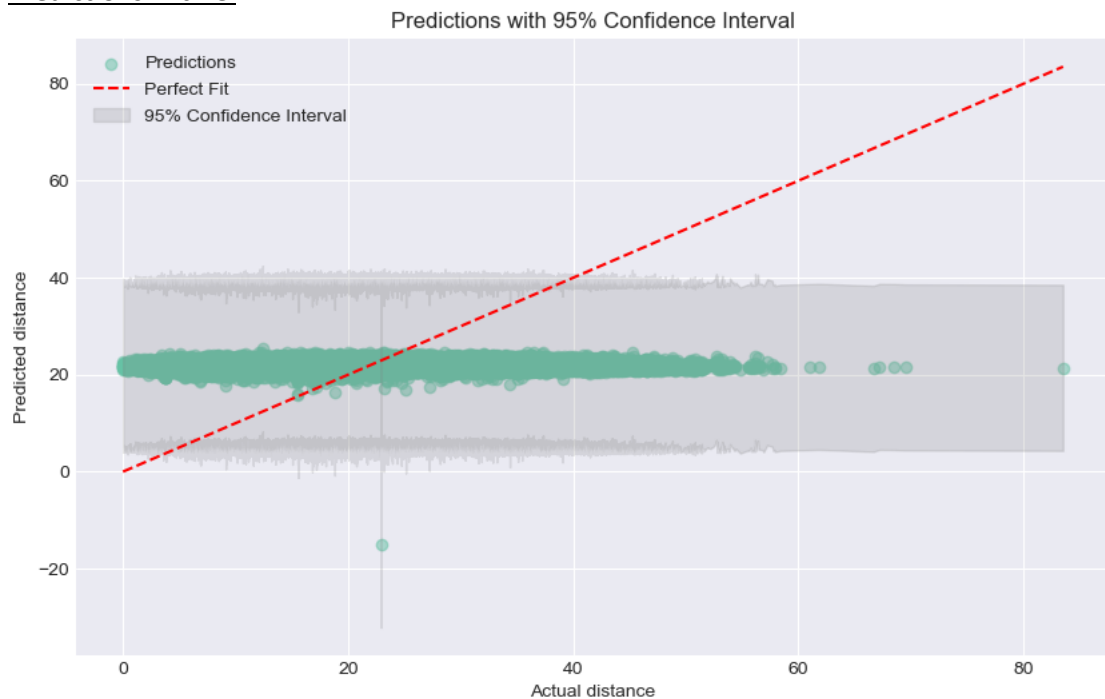
5) **Actual vs Predicted:**



6) **Regression Assumptions:**



7) Predictions with CI:



- 1) Are there any categorical variables in the data? From your analysis of the categorical variables from the dataset, what could you infer about their effect on the dependent variable?
- A) The provided code doesn't explicitly identify or handle categorical variables. The feature preparation function (`prepare_features()`) only selects numeric columns:
- ```
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
```

This suggests that the code is only considering numeric features for the regression analysis. Without seeing the actual data or any categorical variable processing, I can't determine if categorical variables were present in the original dataset or their potential effect on the dependent variable

- 2) What does `test_size = 0.2` refer to during splitting the data into training and test sets?
  - A) In the code, `test_size = 0.3` in the `train_test_split()` function means that 30% of the data will be allocated to the test set, while the remaining 70% will be used for training:  
**`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)`**  
This is a common split ratio that balances having enough data for both training a reliable model and having a sufficiently large test set to evaluate performance
- 3) Looking at the heatmap, which one has the highest correlation with the target variable?
  - A) The code generates a correlation heatmap using `feature_correlations()`, but it doesn't provide the actual correlation values in the script output. The heatmap visualization would show the correlation strengths between all numeric features, but without seeing the actual visualization or the correlation matrix values, I can't determine which feature has the highest correlation with the target variable.
- 4) What was your approach to detect the outliers? How did you address them?
  - A) The provided code doesn't include specific outlier detection or handling methods. The `explore_data()` function does include basic statistics that could help identify outliers (like min, max, and quartiles), but there's no explicit code for outlier detection or removal in the script.
- 5) Based on the final model, which are the top 3 features significantly affecting the delivery time?
  - A) The code sorts the coefficients by magnitude in the `train_linear_regression()` function:  
**`coefficients = coefficients.sort_values(by='Coefficient', ascending=False)`**  
However, the actual output of this sorting isn't provided, so I can't determine which specific features have the highest coefficients. The features with the largest absolute coefficient values would have the most significant effect on the target variable (likely delivery time).
- 6) Explain the linear regression algorithm in detail.
  - A) Linear regression is a statistical modeling technique used to predict a continuous target variable based on one or more predictor variables. It assumes a linear relationship between the predictors and the target. The algorithm works as follows:

**Model Representation:**

- For a single predictor (simple linear regression):  $y = \beta_0 + \beta_1 x + \epsilon$
- For multiple predictors (multiple linear regression):  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$

Where:

- $y$  is the target variable
- $x_1, x_2, \dots, x_n$  are predictor variables
- $\beta_0$  is the intercept (constant term)
- $\beta_1, \beta_2, \dots, \beta_n$  are coefficients representing the relationship strength between each predictor and the target
- $\epsilon$  is the error term (residual)

**Estimation Method:** The goal is to find the values of  $\beta_0, \beta_1, \dots, \beta_n$  that minimize the sum of squared residuals (Ordinary Least Squares or OLS)

This minimization has a closed-form solution using matrix algebra

**Assumptions:**

- Linearity: The relationship between predictors and target is linear
- Independence: Observations are independent of each other
- Homoscedasticity: Residuals have constant variance across all levels of predictors
- Normality: Residuals follow a normal distribution

- No or little multicollinearity: Predictors aren't highly correlated with each other

**Evaluation:** Common evaluation metrics include:

- Mean Squared Error (MSE): Average of squared differences between predictions and actual values
- Root Mean Squared Error (RMSE): Square root of MSE, in the same units as the target
- R-squared ( $R^2$ ): Proportion of variance in the target explained by the model
- Adjusted  $R^2$ : Modified  $R^2$  that accounts for the number of predictors

The code implements these steps through the `train_linear_regression()` function and checks assumptions through the `check_assumptions()` function.

- 7) Explain the difference between simple linear regression and multiple linear regression

**Simple Linear Regression:**

- Uses only one predictor variable ( $x$ ) to predict a target variable ( $y$ )
- Represented as:  $y = \beta_0 + \beta_1 x + \epsilon$
- The relationship is visualized as a straight line on a 2D plot
- Easier to interpret and visualize

**Multiple Linear Regression:**

- Uses two or more predictor variables ( $x_1, x_2, \dots, x_n$ ) to predict a target variable ( $y$ )
- Represented as:  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$
- The relationship is visualized as a hyperplane in a multi-dimensional space
- Can capture more complex relationships and potentially provide better predictions
- May require additional analysis for feature selection and handling multicollinearity
- More difficult to visualize directly

The provided code implements multiple linear regression since it can handle multiple features:

*`feature_cols = numeric_cols[:-1]` # All numeric columns except the last one*

- 8) What is the role of the cost function in linear regression, and how is it minimized?

The cost function measures how well a linear regression model fits the data. It quantifies the difference between predicted values and actual values. In linear regression, the most common cost function is the Mean Squared Error (MSE):

$$MSE = (1/n) * \sum (y_{actual} - y_{predicted})^2$$

**Minimization Process:**

- 1) **Ordinary Least Squares (OLS):** Analytically solves for coefficients that minimize the sum of squared residuals. Has a closed-form solution:  $\beta = (X^T X)^{-1} X^T y$   
Used in the `LinearRegression()` class from scikit-learn
- 2) **Gradient Descent (alternative method):** Iteratively updates coefficients in the direction of steepest decrease of the cost function. Takes steps proportional to the negative gradient of the cost function. Useful when dealing with large datasets where OLS is computationally expensive.

The scikit-learn implementation used in the code likely uses the OLS method for finding the optimal coefficients.

- 9) Explain the difference between overfitting and underfitting.

- A) **Overfitting:** The model performs very well on training data but poorly on unseen (test) data

The model captures noise and random fluctuations in the training data. Signs include:

- High training  $R^2$  but low-test  $R^2$
- Large gap between training and test errors
- Complex model with many features

**Solutions:**

- Regularization (L1/L2 penalties)

- Feature selection
- Cross-validation
- Collecting more data

Underfitting: The model performs poorly on both training and test data. The model is too simple to capture the underlying pattern in the data. Signs include:

- Low  $R^2$  on both training and test sets
- High bias in predictions
- Similar but poor performance on training and test data

Solutions:

- Add more features
- Use more complex models
- Reduce regularization strength
- Feature engineering

In the code, overfitting or underfitting could be detected by comparing training and test set performance metrics:

```
print(f"Training R2: {train_r2:.4f}")
print(f"Testing R2: {test_r2:.4f}")
```

10) How do residual plots help in diagnosing a linear regression model?

A) Residual plots are powerful diagnostic tools that help verify linear regression assumptions and identify potential issues:

- Residuals vs. Predicted Values Plot:**  
 Helps check the linearity and homoscedasticity assumptions  
 Ideally shows random scatter around the zero line  
 Patterns like curves suggest non-linearity  
 Funnel shapes suggest heteroscedasticity (non-constant variance)
- Q-Q Plot of Residuals:**  
 Checks the normality assumption  
 Points should follow the diagonal line if residuals are normally distributed  
 Deviations suggest non-normality
- Residuals Histogram:**  
 Another way to check normality  
 Should approximately follow a bell curve
- Residuals vs. Actual Values Plot:**  
 Helps check for patterns related to the magnitude of the target variable  
 Should show random scatter with consistent spread